栈和队列全面笔记

目录

1. 栈
2. 队列
3. 优先级队列
4. 栈与递归
5. 表达式计算的优化：双栈同时计算
6. 常考题型与易错点

一、栈 (Stack)

1.1 栈的基本概念

栈（Stack） 是一种只允许在**一端**进行插入和删除操作的线性表。

```
        ←── 进栈（push）
        ←── 出栈（pop）
        ┌─────┐
        │ ... │  ← 栈顶（top）
        │ ... │
        │ ... │
        └─────┘
          ↑
       栈底（bottom）
```

术语	含义
栈顶（top）	允许插入/删除的一端
栈底（bottom）	不允许操作的一端
空栈	没有任何元素的栈
上溢（overflow）	栈已满时还做入栈操作
下溢（underflow）	栈为空时还做出栈操作

核心特性：后进先出（LIFO, Last In First Out）

> **适于处理具有递归结构的数据**（如函数调用、表达式求值）

> **生活类比：** 子弹压入弹夹，最后压入的子弹最先打出；一叠盘子，最后放上去的最先被拿走；地窖藏酒，最后放进去的酒最先取出来。

栈与一般线性表的关系： 栈是一种**操作受限的线性表**，插入和删除只能在栈顶进行，是线性表操作的子集。

1.2 栈的基本操作

```
void push(Stack s, ElemType e);   // 进栈（压栈、入栈）
ElemType pop(Stack s);            // 出栈（弹栈、退栈）
int isEmpty(Stack s);             // 判空
int isFull(Stack s);              // 判满
ElemType getTop(Stack s);         // 取栈顶元素（不删除）
void initStack(Stack s);          // 初始化栈
```

> 所有操作的时间复杂度均为 **O(1)**。

1.3 栈的顺序存储（顺序栈）

结构原理

用一维数组存储，`top` 指示栈顶元素的下标位置。

```
#define MAXSIZE 1000
typedef int ElemType;

ElemType stack[MAXSIZE];
int top;
```

两种 top 指针方案对比：

方案	top 初值	栈满条件	进栈操作	出栈操作
top 指向栈顶元素	`-1`	`top == MAXSIZE-1`	`stack[++top] = e`	`e = stack[top--]`
top 指向下一个空位	`0`	`top == MAXSIZE`	`stack[top++] = e`	`e = stack[--top]`

> 考研中常用 **top = -1** 方案，进栈先移指针再赋值，出栈先取值再移指针。

完整实现（top = -1 方案）

```
#define MAXSIZE 1000
ElemType stack[MAXSIZE];
int top = -1;          // 初始化栈空

// 判空
int isEmpty() {
    return top == -1;
}

// 判满
int isFull() {
    return top == MAXSIZE - 1;
}
```

```c
// 进栈
void push(ElemType e) {
    if (isFull()) {
        printf("Stack overflow!\n");
        exit(1);
    }
    stack[++top] = e;  // top 先加 1，再赋值
}

// 出栈
ElemType pop() {
    if (isEmpty()) {
        printf("Stack underflow!\n");
        exit(1);
    }
    return stack[top--];  // 先取值，top 再减 1
}

// 取栈顶
ElemType getTop() {
    if (isEmpty()) {
        printf("Stack is empty!\n");
        exit(1);
    }
    return stack[top];
}
```

进栈/出栈过程示意：

```
初始：  top = -1,  stack = [  ][  ][  ][  ]

push(10) → top 先 +1 → top=0 → stack[0]=10
           stack = [10][  ][  ][  ]

push(20) → top=1 → stack[1]=20
           stack = [10][20][  ][  ]

push(30) → top=2 → stack[2]=30
           stack = [10][20][30][  ]

pop() → 返回 stack[2]=30 → top=1
         stack 数据还在内存中，但逻辑上 top=1 表示只有 [10,20]
```

1.4 栈的链式存储（链栈）

结构原理

用单链表实现，**始终在链表头部操作**。`top` 指向栈顶结点（链表第一个结点），空栈时 `top = NULL`。

```
push(10):  top → [10|/]
push(20):  top → [20|*] → [10|/]
push(30):  top → [30|*] → [20|*] → [10|/]
pop():     返回 30，top → [20|*] → [10|/]（释放 [30] 结点）
```

```c
typedef struct Node {
    ElemType data;
    struct Node *next;
} Node, *NodePtr;

NodePtr top = NULL;   // 空栈

// 判空
int isEmpty() {
    return top == NULL;
}

// 进栈（头插法）
void push(ElemType e) {
    NodePtr p = (NodePtr)malloc(sizeof(Node));
    if (p == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    p->data = e;
    p->next = top;   // 新结点指向原栈顶
    top = p;          // 新结点成为栈顶
}

// 出栈（头删法）
ElemType pop() {
    if (isEmpty()) {
        printf("Stack underflow!\n");
        exit(1);
    }
    NodePtr p = top;
    ElemType e = p->data;
    top = top->next;  // top 指向下一个结点
    free(p);           // 释放原栈顶
    return e;
}

// 取栈顶
ElemType getTop() {
    if (isEmpty()) {
        printf("Stack is empty!\n");
        exit(1);
    }
```

```
    return top->data;
}
```

顺序栈 vs 链栈

对比项	顺序栈	链栈
空间分配	静态分配，大小固定	动态分配，随需增长
溢出风险	可能上溢（数组满）	除非内存耗尽，一般不溢出
操作效率	O(1)，数组直接访问	O(1)，指针操作
空间利用	可能有空闲空间	每结点多存一个指针（额外 4/8 字节）

1.5 栈的应用：表达式计算 ⭐⭐⭐⭐⭐

这是栈最重要的应用，考研必考！

三种表达式表示法

名称	示例	特点
中缀表达式	`a + b * c`	运算符在中间，需括号和优先级，**人类习惯**
后缀表达式（逆波兰 RPN）	`a b c * +`	运算符在后面，**无括号、无优先级，机器最爱**
前缀表达式（波兰式）	`+ a * b c`	运算符在前面

中缀 → 后缀转换算法

核心思想：用栈暂存运算符，根据优先级决定何时输出。

```
遍历中缀表达式的每个符号：
  ├─ 如果是数字 → 直接输出
  ├─ 如果是 '(' → 入栈
  ├─ 如果是 ')' → 依次弹出栈中运算符并输出，直到遇到 '('
  │               （ '(' 弹出但不输出）
  └─ 如果是运算符 → 弹出栈中所有优先级 ≥ 当前运算符的符号并输出
                    （遇到 '(' 停止），再将当前运算符入栈
遍历结束后 → 弹出栈中所有剩余运算符并输出
```

手动模拟示例：`a + b * c - d`

```
符号   栈       输出        说明
a               a           数字直接输出
+     [+]       a           栈空，直接入栈
b     [+]       a b         数字直接输出
*     [+ *]     a b         优先级 * > +，入栈
```

```
c     [+ *]      a b c      数字直接输出
-     [-]        a b c * +  优先级 - ≤ *，弹出 *；优先级 - ≤ +，弹出 +；栈空，入栈 -
d     [-]        a b c * + d
结束               a b c * + d -  弹出 -
```

带括号示例： `a * (b + c)`

```
符号  栈          输出         说明
a                  a           数字直接输出
*     [*]          a           栈空，入栈
(     [* (]        a           '(' 直接入栈
b     [* (]        a b         数字直接输出
+     [* ( +]      a b         栈顶是 '('，+ 直接入栈
c     [* ( +]      a b c       数字直接输出
)     [*]          a b c +     弹出 + 直到 '('，'(' 弹出不输出
结束                 a b c + *  弹出 *
```

优先级表：

运算符	优先级值	说明
`* /`	1	高优先级
`+ -`	0	低优先级
`(`	—	特殊，入栈后仅当遇到 `)` 才弹出

后缀表达式计算算法

```
遍历后缀表达式的每个符号：
  ├─ 如果是数字 → 入栈
  └─ 如果是运算符 → 弹出两个操作数计算，结果入栈
                    ⚠️ 先弹出的是右操作数，后弹出的是左操作数
遍历结束 → 栈中唯一元素就是结果
```

示例： `5 2 3 * +`

```
符号  栈（从底到顶）     操作
5     [5]
2     [5, 2]
3     [5, 2, 3]
*     [5, 6]         2 * 3 = 6
+     [11]           5 + 6 = 11  ← 结果
```

优化：中缀转后缀与计算同时进行（双栈法）

实际实现时，**不需要先完整转成后缀再计算**，可以设**数据栈**和**运算符栈**，在转换后缀的同时进行计算：

```
遍历中缀表达式：
  遇到数字 → 压入数据栈
  遇到运算符 → 与运算符栈顶比较优先级：
    ├─ 当前运算符优先级 > 栈顶 → 入运算符栈
    └─ 当前运算符优先级 ≤ 栈顶 → 弹出运算符栈顶，
       同时从数据栈弹出两个数据进行计算，结果压回数据栈
       （循环直到当前运算符可以入栈）
  遇到 ')' → 弹出运算符并计算，直到遇到 '('
遍历结束 → 依次弹出运算符并计算
最终数据栈栈顶 = 结果
```

核心思想： 每个运算符出栈时，立刻用数据栈中的数据进行计算，计算结果存回数据栈。

```
以 2 + 3 * 4 为例：

符号   数据栈      运算符栈     操作
2      [2]          []
+      [2]          [+]         运算符栈空，入栈
3      [2,3]        [+]
*      [2,3]        [+ *]       优先级 * > +，入栈
4      [2,3,4]      [+ *]
结束    → 弹出 * → 计算 3*4=12 → 数据栈 [2,12]
        → 弹出 + → 计算 2+12=14 → 数据栈 [14]  ← 结果
```

对比之前的"分两步"方法，这种做法**效率更高**（只需遍历一遍），课堂上提供的完整代码就是用的这种双栈同时计算方法。

> ⚠️ **减法和除法要注意操作数顺序：**
>
> - 后缀 `a b -` 计算 `a - b`（先弹出的是 b，后弹出的是 a）
> - 后缀 `a b /` 计算 `a / b`
>
> 代码实现：
>
> ```
> case MIN:
>  tmp = popNum();            // 先弹出的是右操作数 b
>  pushNum(popNum() - tmp);   // 后弹出的是左操作数 a
>  break;
> ```

原版课堂代码的两处错误修正

错误 1：`pushOp` 函数判满用了错误的栈顶指针

```
// ❌ 原文错误
void pushOp(enum oper op) {
    if (Ntop == MAXSIZE - 1) {    // 应该判断 Otop 而不是 Ntop!
        printf("operator stack is full!\n");
        exit(1);
    }
    Op_stack[++Otop] = op;
}

// ✅ 修正
void pushOp(enum oper op) {
    if (Otop == MAXSIZE - 1) {
        printf("operator stack is full!\n");
        exit(1);
    }
    Op_stack[++Otop] = op;
}
```

错误 2：`topOp()` 在栈空时访问 `Op_stack[-1]` 造成数组越界

```
// ❌ 原文问题
enum oper topOp() {
    return Op_stack[Otop];   // 当 Otop = -1 时访问了 Op_stack[-1]，越界!
}

// ✅ 修正
enum oper topOp() {
    if (Otop == -1)          // 栈空时返回 EPT（空运算符，优先级最低）
        return EPT;
    return Op_stack[Otop];
}
```

1.6 应用：括号匹配 ⭐⭐⭐

题目： 检查表达式中的括号是否正确匹配。

算法：

```
遍历每个字符：
  ├─ 遇到左括号 '(' '[' '{' → 入栈
  ├─ 遇到右括号 → 若栈空则失配；否则弹出栈顶，检查是否匹配
  └─ 其他字符 → 跳过
遍历结束 → 栈空则全部匹配，栈不空则有多余左括号
```

```
int isMatching(char left, char right) {
    return (left == '(' && right == ')') ||
```

```
            (left == '[' && right == ']') ||
            (left == '{' && right == '}');
}

int checkBrackets(char expr[]) {
    char stack[1000];
    int top = -1;

    for (int i = 0; expr[i] != '\0'; i++) {
        if (expr[i] == '(' || expr[i] == '[' || expr[i] == '{') {
            stack[++top] = expr[i];          // 左括号入栈
        } else if (expr[i] == ')' || expr[i] == ']' || expr[i] == '}') {
            if (top == -1) return 0;          // 栈空，无匹配左括号
            if (!isMatching(stack[top--], expr[i])) return 0;  // 不匹配
        }
    }
    return top == -1;  // 栈空则全部匹配
}
```

二、队列 (Queue)

2.1 队列的基本概念

队列（Queue） 是一种只允许在**队尾插入**、**队头删除**的线性表。

```
出队 ← [ 队头 | 数 | 数 | 数 | 队尾 ] ← 入队
        Front                       Rear
```

术语	含义
队头（Front）	允许删除的一端
队尾（Rear）	允许插入的一端
空队	没有元素的队列
假溢出	数组空间未满，但 Rear 已到末尾，无法再入队

核心特性：先进先出（FIFO, First In First Out）

> **适于存放需要按一定次序依次处理但尚未处理的数据**（如排队系统、缓冲区、任务调度）

> **生活类比：** 食堂排队打饭，先到先打；银行排队叫号，先取号先服务。

2.2 队列的基本操作

```
void enQueue(Queue q, ElemType e);   // 入队（进队）
ElemType deQueue(Queue q);            // 出队（退队）
int isEmpty(Queue q);                 // 判空
```

```
int isFull(Queue q);                    // 判满
ElemType getFront(Queue q);             // 取队头元素
void initQueue(Queue q);                // 初始化队列
```

2.3 队列的顺序存储

为什么需要循环队列?

普通顺序队列存在 **假溢出** 问题:

```
初始:  Front=0, Rear=3
       [a][b][c][d][ ][ ]
        ↑       ↑
      Front   Rear

出队两次后:  Front=2
       [a][b][c][d][ ][ ]
              ↑    ↑
           Front Rear

再入队:  Rear 已经到末尾了! 但前面还有空位!
         这就是"假溢出"—数组没满但存不了新元素
```

循环队列把数组在逻辑上首尾相连，通过取模运算 `(Rear + 1) % MAXSIZE` 让 Rear 能从末尾绕回开头。

实现方案一：计数法（Count）

最推荐，最简单直观，不易出错。

```
#define MAXSIZE 1000
typedef int ElemType;

ElemType queue[MAXSIZE];
int front = 0;           // 队头下标
int rear = MAXSIZE - 1;  // 队尾下标（初始在末尾，第一次入队时通过取模回到 0）
int count = 0;           // 当前元素个数

// 判空
int isEmpty() { return count == 0; }

// 判满
int isFull() { return count == MAXSIZE; }

// 入队
void enQueue(ElemType e) {
    if (isFull()) { printf("Queue full!\n"); exit(1); }
    rear = (rear + 1) % MAXSIZE;   // rear 后移（循环绕回）
    queue[rear] = e;
    count++;
```

```
}

// 出队
ElemType deQueue() {
    if (isEmpty()) { printf("Queue empty!\n"); exit(1); }
    ElemType e = queue[front];
    front = (front + 1) % MAXSIZE; // front 后移（循环绕回）
    count--;
    return e;
}
```

过程示意（MAXSIZE=5）：

```
初始:  front=0, rear=4, count=0
       [  ][  ][  ][  ][  ]

enQueue(10):  rear=(4+1)%5=0, count=1
       [10][  ][  ][  ][  ]
       ↑
    front&rear

enQueue(20):  rear=1, count=2
       [10][20][  ][  ][  ]
        ↑   ↑
      front rear

deQueue():  返回10, front=1, count=1
       [10][20][  ][  ][  ]
            ↑
        front&rear

enQueue(30):  rear=2, count=2
enQueue(40):  rear=3, count=3
enQueue(50):  rear=4, count=4
       [10][20][30][40][50]
            ↑           ↑
          front        rear

enQueue(60):  rear=(4+1)%5=0, count=5（绕回了！）
       [60][20][30][40][50]
        ↑    ↑
       rear front
```

实现方案二：牺牲一个存储单元

不用 count，通过判断 `(Rear + 1) % MAXSIZE == Front` 知满。

```
int front = 0;  // 队头
int rear = 0;   // 队尾（指向下一个空位）
```

```
// 判空：front == rear
int isEmpty() { return front == rear; }

// 判满：(rear + 1) % MAXSIZE == front
int isFull() { return (rear + 1) % MAXSIZE == front; }

// 入队
void enQueue(ElemType e) {
    if (isFull()) exit(1);
    queue[rear] = e;
    rear = (rear + 1) % MAXSIZE;
}

// 出队
ElemType deQueue() {
    if (isEmpty()) exit(1);
    ElemType e = queue[front];
    front = (front + 1) % MAXSIZE;
    return e;
}
```

> ⚠️ 牺牲单元法始终有一个空位不能用，实际最多存 **MAXSIZE-1** 个元素。

2.4 循环队列三种判空/判满方案对比

方案	判空条件	判满条件	最大元素数	难易度
计数法（Count）	`Count == 0`	`Count == MAXSIZE`	MAXSIZE	⭐ 最简单
牺牲单元法	`Front == Rear`	`(Rear+1)%MAXSIZE == Front`	MAXSIZE-1	⭐⭐⭐ 常见
增设标志位 tag	`Front==Rear && tag==0`	`Front==Rear && tag==1`	MAXSIZE	⭐⭐ 少见

> 考研中最常考 **牺牲单元法**，但 **计数法最不易出错**，实际编程推荐计数法。

2.5 队列的链式存储（链队）

结构原理

用单链表实现，**队头删除**（头删）、**队尾插入**（尾插）。

```
队列：  front → [a1|*] → [a2|*] → [a3|/]
                                    ↑
                                  rear
```

```c
typedef struct QNode {
    ElemType data;
    struct QNode *next;
} QNode, *QNodePtr;

QNodePtr front = NULL;  // 队头指针
QNodePtr rear = NULL;   // 队尾指针

// 初始化
void initQueue() {
    front = rear = NULL;
}

// 判空
int isEmpty() {
    return front == NULL;  // 或 rear == NULL
}

// 入队（尾插法）
void enQueue(ElemType e) {
    QNodePtr p = (QNodePtr)malloc(sizeof(QNode));
    p->data = e;
    p->next = NULL;

    if (isEmpty()) {
        front = p;          // 第一个结点
    } else {
        rear->next = p;     // 原尾结点指向新结点
    }
    rear = p;               // 更新尾指针
}

// 出队（头删法）
ElemType deQueue() {
    if (isEmpty()) {
        printf("Queue empty!\n");
        exit(1);
    }
    QNodePtr p = front;
    ElemType e = p->data;

    front = front->next;    // 队头后移
    if (front == NULL) {    // 队列空了，尾指针也要置空
        rear = NULL;
    }
    free(p);
    return e;
}
```

顺序队列 vs 链队

对比项	顺序队列（循环）	链队
空间	固定大小	动态增长
操作	取模运算	指针操作
判满	需要判满	无需判满（malloc 失败除外）
空队判断	Count==0 或 Front==Rear	Front==NULL

2.6 队列的应用：银行排队模拟

问题描述

模拟银行对私业务排队，动态调整服务窗口数（3~5 个），输出每个客户的等待时间。

核心流程

```
每个时间周期（clock）：
  1. updateCustqueue() — 所有在队列中的客户等待时间 +1
  2. arriveCust()      — 新客户到达，加入队列
     ├─ 读入本周期到达的客户数
     └─ 如果排队人数太多 → 增加窗口（最多 5 个）
  3. service()         — 每个窗口服务一个客户（出队）
     ├─ 打印客户 ID 和等待时间
     └─ 如果排队人数较少 → 减少窗口（最少 3 个）
```

动态调整窗口的逻辑：

```
// 平均每个窗口排队人数 = 当前队列长度 / 窗口数

// 排队太长 → 加窗口
if (队列长度 / 窗口数 >= 阈值 && 窗口数 < 最大窗口数)
    窗口数++;

// 排队太短 → 减窗口
if (队列长度 / 窗口数 < 阈值 && 窗口数 > 最小窗口数)
    窗口数--;
```

代码中的注意点： `cutqueue.c` 用 `static` 关键字将队列数组和指针隐藏在文件内部，实现了**数据隐藏**，只对外暴露操作函数——这是 C 语言实现封装的常用技巧。

还有一个细节要注意： 原代码中变量名不一致——全局变量用的是 `Crear`（应为 `Rear`），初始化 `Crear = -1` 等价于之前讲的 `Rear = MAXSIZE - 1`（第一次入队时 `(-1+1)%MAXSIZE = 0`）。

银行排队模拟的改进：优先级规则

原策略中"平均排队人数 < 阈值就减少窗口"并不符合实际。更合理的规则：

改进规则：

1. **外币窗口和对公窗口优先处理本业务**——当有外汇/对公客户等待时，优先处理
2. **空闲窗口转做对私**——只有当外币/对公窗口没有本业务排队客户时，才处理对私业务
3. **资源利用最大化**——所有窗口在没有本业务客户时，自动转为对私服务

```
每个时间周期：
  1. 外币窗口：有外币客户？→ 处理外币客户；否则处理对私客户
  2. 对公窗口：有对公客户？→ 处理对公客户；否则处理对私客户
  3. 对私窗口：始终处理对私客户
```

这种策略实际上就是**优先级队列**的思想——不同业务的客户有不同的优先级，但同一业务内部仍按 FIFO 排队。

三、优先级队列

3.1 概念

优先级队列（Priority Queue） 不再遵循 FIFO，而是**按优先级高低**决定出队顺序（优先级最高的先出队）。

3.2 应用场景

场景	优先级规则
医院急诊	危重病人先就诊（病情优先级）
操作系统进程调度	优先级高的进程先获得 CPU
急救车通行	优先通过路口
银行 VIP 客户	VIP 客户优先办理

3.3 实现方式对比

实现方式	入队时间复杂度	出队时间复杂度	说明
普通链表（插末尾）	O(1)	O(n)（需遍历找最高优先级）	出队慢
有序链表（按优先级排序）	O(n)（找位置插入）	O(1)（取头部）	入队慢
堆（Heap）	**$O(\log_2 n)$**	**$O(\log_2 n)$**	效率最平衡，最常用

四、栈与递归

4.1 递归的栈式执行过程

递归调用时，系统使用**函数调用栈（call stack）** 保存每层调用的信息：

```
每次函数调用 → 栈帧入栈（保存返回地址、参数、局部变量）
```

```
每次函数返回 → 栈帧出栈（恢复上一层现场）
```

以阶乘 `fact(3)` 为例：

```
调用过程（入栈）：
  main 调用 fact(3): 保存返回地址，压入参数 3
    fact(3)调用 fact(2): 保存返回地址，压入参数 2
      fact(2)调用 fact(1): 保存返回地址，压入参数 1
        fact(1) → 直接返回 1

返回过程（出栈）：
      fact(1)返回 1，弹出栈帧，恢复 fact(2) 的现场
    fact(2)计算 2*1=2，返回，弹出，恢复 fact(3) 的现场
  fact(3)计算 3*2=6，返回，弹出，恢复 main 的现场
```

4.2 递归转非递归

核心思想： 用程序员**自定义的栈**来模拟系统栈。

练习题目给的例子：

```c
// 递归版本
void process(int n) {
    if (n > 1) {
        printf("%d ", n);
        process(n - 1);
    }
}
// 调用 process(5) → 输出 5 4 3 2
```

```c
// 非递归版本（用栈模拟）
void process_nonrecursive(int n) {
    int stack[1000], top = -1;

    // 将 n 到 2 依次压栈（模拟递归调用的参数入栈）
    while (n > 1) {
        stack[++top] = n;
        n--;
    }

    // 出栈并打印（模拟递归返回时的回溯输出）
    while (top >= 0) {
        printf("%d ", stack[top--]);
    }
}
```

4.3 汉诺塔

```
void hanoi(int n, char x, char y, char z) {
    // 将 n 个盘子从 x 移到 z，借助 y
    if (n > 0) {
        hanoi(n - 1, x, z, y);           // ① n-1 个从 x→y（借助 z）
        printf("MOVE %d: %c -> %c\n", n, x, z);  // ② 最大盘从 x→z
        hanoi(n - 1, y, x, z);           // ③ n-1 个从 y→z（借助 x）
    }
}
```

> **移动次数：** $2^n - 1$。当 n = 64 时约 1844 亿亿次（传说中世界末日的那一天）。

六、常考题型与易错点

5.1 出栈序列合法性 ⭐⭐⭐⭐⭐

题目： 入栈序列为 `1,2,3,...,n`，判断一个出栈序列是否合法。

判断方法： 模拟出入栈过程。

```
int isValidPopSequence(int push[], int pop[], int n) {
    int stack[1000], top = -1;
    int pi = 0, pj = 0;   // pi 指向 push，pj 指向 pop

    while (pi < n) {
        stack[++top] = push[pi++];  // 入栈

        // 栈顶等于当前出栈元素 → 出栈
        while (top >= 0 && stack[top] == pop[pj]) {
            top--;
            pj++;
        }
    }
    return top == -1;  // 栈空则全部匹配
}
```

示例： 入栈 1,2,3,4,5，判断出栈 3,2,4,1,5 是否合法？

```
出栈序列: 3 2 4 1 5

push 1 → stack [1], 栈顶=1 ≠ 3
push 2 → stack [1,2], 栈顶=2 ≠ 3
push 3 → stack [1,2,3], 栈顶=3 = 3 → pop → stack [1,2], pj→1(2)
栈顶=2 = 2 → pop → stack [1], pj→2(4)
push 4 → stack [1,4], 栈顶=4 = 4 → pop → stack [1], pj→3(1)
栈顶=1 = 1 → pop → stack [], pj→4(5)
```

```
push 5 → stack [5], 栈顶=5 = 5 → pop → stack [], pj→5
→ 合法 ✅
```

快速判断规律（312 法则）： 对出栈序列中的任意三个位置 i<j<k，若出现 `p[i] > p[k] > p[j]`（即 312 模式），则非法。

5.2 求最小栈容量 ⭐⭐⭐⭐

题目： 入栈序列 `a,b,c,d,e,f`，出栈顺序 `b,d,c,f,e,a`，判断是否合法？若合法，过程中栈的容量至少为多少？

解法： 模拟出入栈过程，同时记录栈的最大深度。

```c
int minStackCapacity(char push[], char pop[], int n) {
    char stack[1000];
    int top = -1;
    int pi = 0, pj = 0;
    int max_cap = 0;  // 记录栈的最大深度

    while (pi < n) {
        stack[++top] = push[pi++];
        if (top + 1 > max_cap) max_cap = top + 1;  // 更新最大深度

        while (top >= 0 && stack[top] == pop[pj]) {
            top--;
            pj++;
        }
    }
    return (top == -1) ? max_cap : -1;  // 合法返回最小容量，非法返回 -1
}
```

示例验证： 入栈 `a,b,c,d,e,f`，出栈 `b,d,c,f,e,a`

```
操作        栈（底→顶）   当前容量    出栈
push a     [a]          1
push b     [a,b]        2
pop b      [a]          2          b
push c     [a,c]        2
push d     [a,c,d]      3
pop d      [a,c]        3          d
pop c      [a]          3          c
push e     [a,e]        3
push f     [a,e,f]      3
pop f      [a,e]        3          f
pop e      [a]          3          e
pop a      []           3          a

→ 合法，最小栈容量 = 3 ✅（选项 B）
```

快速判断技巧： 当出栈序列中出现"大数在小数之后出栈，但大数先于某个更小的数入栈"的嵌套情况时，需要更大的栈容量。容量取决于**过程中栈的最大深度**。

5.3 卡特兰数

n 个不同元素进栈，可能的出栈序列总数 = **卡特兰数**：

```
C(n) = (2n)! / (n! · (n+1)!)
```

n	C(n)	说明
1	1	
2	2	12, 21
3	**5**	123, 132, 213, 231, **321**（312 不合法！）
4	14	
5	42	

5.3 栈和队列的相互模拟

用两个栈模拟队列：

```
入队：元素压入栈 A
出队：若栈 B 非空，从 B 弹出；若 B 空，将 A 中所有元素弹出压入 B，再从 B 弹出

示例：
enQueue(1), enQueue(2), enQueue(3):
  A = [1,2,3]（底→顶）, B = []

deQueue():
  A 全部弹出压入 B → A = [], B = [3,2,1]
  从 B 弹出 1 ← 就是最早入队的元素 ✅
```

用两个队列模拟栈：

```
入栈：元素进入非空队列（初始任选一个）
出栈：将非空队列中除最后一个元素外全部移到另一个队列，剩下的就是栈顶
```

5.4 两栈串联问题 ⭐⭐⭐⭐

题目： 两个栈 S1、S2 串联，元素从 S1 顶部进入，从 S2 顶部输出，如下图所示。元素**不能反向运动**（即一旦从 S1 弹出进入 S2，不能再回到 S1）。问：n 个不同元素依次进入 S1，可能的输出序列有多少种？

```
进入 → [S1 栈] → 弹出 → [S2 栈] → 输出
        ┌───┐            ┌───┐
        │   │            │   │ ← 输出
        │   │  →pop→   │   │
        │   │  →push→  │   │
        └───┘            └───┘
```

分析： 这就是**两个栈串联模拟队列**的模型（元素只能单向运动）。

- S1 负责接收输入（相当于入队）
- S2 负责输出（相当于出队）
- 当 S2 为空时，必须将 S1 中所有元素弹出并压入 S2，然后从 S2 输出

结论： 这种结构实际模拟的就是一个**队列**的 FIFO 行为。当所有元素进入 S1 后，再全部转移到 S2，输出顺序与输入顺序**完全一致**（FIFO）。

如果允许多次交错转移（即 S1 随时可以转移一批到 S2），则输出序列由 S2 空出的时机决定，可能的输出序列数 = **卡特兰数**。

> **扩展思考：** 三个及以上的栈串联呢？会生成全部排列吗？这是栈排序理论中的一个经典问题。

5.5 易错点总结

易错点	说明
顺序栈 top 初值	top = -1（指向栈顶）还是 top = 0（指向空位），操作完全不同
循环队列判满	牺牲单元法 `(R+1)%M == F` 和计数法 `Count == M` 不要混淆
后缀表达式操作数顺序	减法和除法：先弹出的是右操作数，后弹出的是左操作数
链队出队后尾指针	队列变空时要将 rear 也置 NULL，否则 rear 成为悬空指针
递归转非递归	注意保存的不只是参数，还有返回地址和局部变量

5.5 核心口诀

```
栈是后进先出，队列先进先出。
栈顶进出 O(1)，队头出队尾入。
顺序栈 top 定位置，进先移再放，出先取再移。
循环队列用取模，计数法最省心。
中缀转后缀用栈，后缀计算也用栈。
出栈序列卡特兰数，312 模式记清楚。
两个栈能模拟队，两个队列能模拟栈。
```